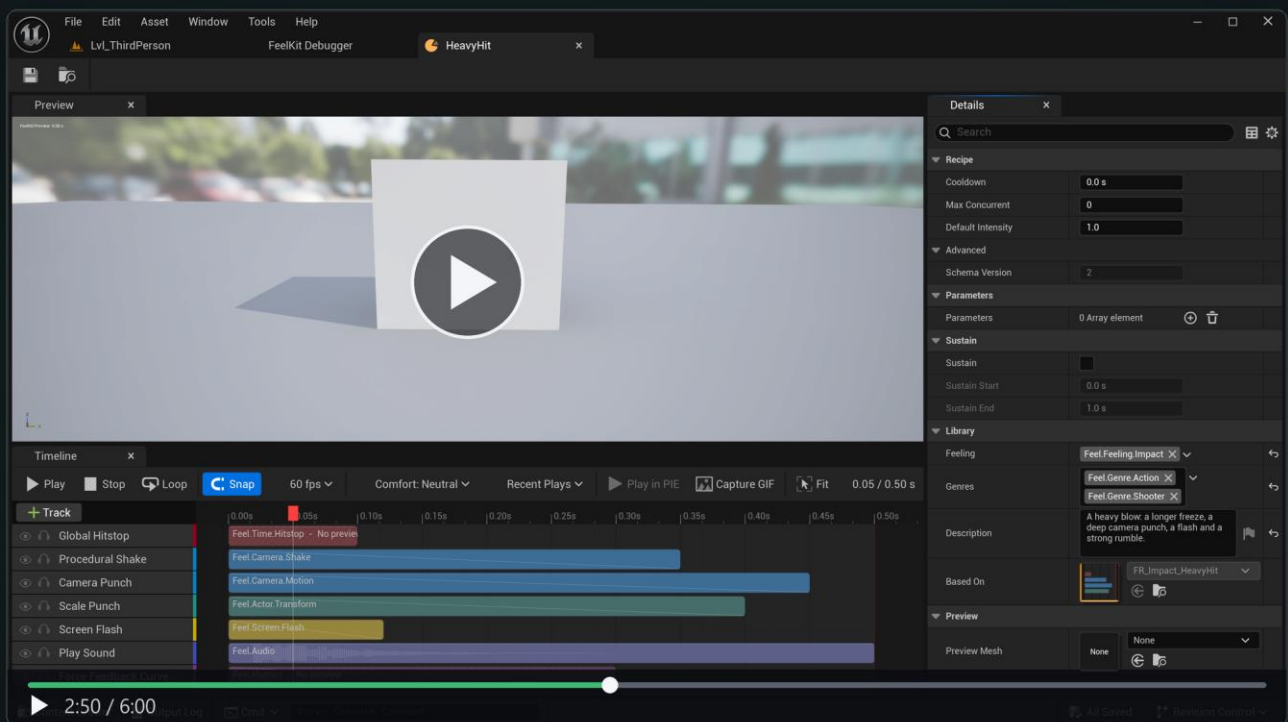


UNREAL ENGINE PLUGIN

FeelKit

Video Guide

Recording, editing and publishing the tutorials and the listing videos, shot by shot.



SETUP

OBS, microphone, Windows and Unreal, on this machine.

SCRIPTS

Four tutorials and two listing videos, with every word.

PUBLISHING

Editing, captions, loudness, YouTube and the Fab gallery.

Contents

1 The videos	4
1.1 The set	4
1.2 Why these lengths	5
2 Recording setup.....	6
2.1 This machine	6
2.2 Software	7
2.3 Capture resolution and application scale.....	7
2.4 OBS profile, scene and sources	8
2.5 Microphone and room.....	9
2.6 Windows preparation.....	10
2.7 Unreal preparation.....	10
2.8 Pre-recording checklist.....	11
3 Narration and screen style.....	13
3.1 Voice and wording.....	13
3.2 Pointer and keyboard.....	13
3.3 Takes and the take log.....	14
4 Video 1: Install FeelKit and play your first recipe.....	15
4.1 Setup	15
4.2 Script	16
5 Video 2: Build a recipe on the timeline	19
5.1 Setup	19
5.2 Script	20
6 Video 3: Trigger recipes from your game.....	23
6.1 Setup	23
6.2 Script	24
7 Video 4: Comfort settings for your players.....	27
7.1 Setup	27

7.2 Script	28
8 Listing videos	31
8.1 T1: FeelKit Pro	32
8.2 T2: FeelKit Lite.....	33
9 Editing.....	34
9.1 Project and folders.....	34
9.2 Timeline structure	34
9.3 Zoom-ins	35
9.4 Callouts and text.....	35
9.5 Captions	36
9.6 Loudness and mix	36
9.7 Music.....	37
9.8 Feel off and feel on.....	37
10 Export and publishing.....	38
10.1 Export settings	38
10.2 YouTube upload.....	39
10.3 Where the links go	39
11 Quality control and schedule	41
11.1 Checks for every video	41
11.2 Checks per video.....	42
11.3 Production schedule.....	42
12 What can be automated	43
12.1 Smooth frames at a fixed time step	43
12.2 Identical takes for the comparisons	44
12.3 Split between the developer and the person recording	44

PRO A video or shot that shows FeelKit Pro. Everything without this label applies to Lite and Pro.

Draft, 6 October 2026

1 The videos

IN THIS CHAPTER

1.1 The set

1.2 Why these lengths

The tutorials repeat the manual's steps in motion for readers who would rather watch, and each one belongs to one chapter of the manual, which embeds it as a video box. The listing videos answer a different question: what FeelKit changes in a game, shown on the Fab page in under 90 seconds.

1.1 The set

ID	Title	Where it is used	Audience	Length	Edition
V1	Install FeelKit and play your first recipe	Manual 3.2 and 4.1; YouTube playlist; both listing descriptions	New owners of Lite or Pro, Blueprint users	6 min	Lite and Pro. One shot names the Recipe Browser with a PRO label
V2	Build a recipe on the timeline	Manual 5.1 and 7.1	Designers and programmers who author recipes	9 min	Lite and Pro. The parameter shot and the waveform are Pro
V3	Trigger recipes from your game	Manual 6.1, 6.5, 6.6 and 6.7	Blueprint users who connect gameplay to recipes	9 min	PRO Everything after Play Feel and Stop Feel is Pro
V4	Comfort settings for your players	Manual 8.1 and 8.3	Developers preparing accessibility options	7 min	Lite and Pro. The Debugger and Comfort Audit shots are Pro
T1	FeelKit Pro listing video	Fab gallery of the Pro listing; YouTube	Fab visitors comparing products	75 s	PRO Shows the four demos, which use Pro features

ID	Title	Where it is used	Audience	Length	Edition
T2	FeelKit Lite listing video	Fab gallery of the Lite listing	Fab visitors looking for a free option	40 s	Lite. Shows only what the Lite download contains

Sources for the listing videos: the Launch document allows a gallery video (1920 x 1080, up to 300 MB, MP4, MOV or WEBM) and names a feel off / feel on clip as a candidate; the marketing brief asks for video scripts and marks every visual as real; the Product document lists trailer material among the content. The video table above feeds the manual's video boxes and holds only V1 to V4; the T1 and T2 links go into the Launch document (section 4.1).

Not covered here: the one "how I made it" walkthrough of the Action/RPG sword hit. It is Appendix B of the manual, planned as a document with screenshots and marked Later.

1.2 Why these lengths

- **One task per video.** A tutorial viewer follows along in their own editor and pauses often. Each video covers one manual chapter, so a viewer who needs one task watches one video. All four stay under 10 minutes; short, single-task videos are easier to review and to find again.
- **Time per action.** An action that the viewer copies (point at the control, click, see the result) needs 20 to 45 seconds at a pace that can be followed without pausing. V1 has 15 such actions plus an opening result and an end card (about 6 minutes). V2 has 13 actions, several with two results each (about 9 minutes). V3 covers six ways to play a recipe at 60 to 90 seconds each (about 9 minutes). V4 has 10 comfort topics at 30 to 60 seconds each (about 7 minutes).
- **Narration density.** At 140 words per minute, the scripts need 4.4 minutes of speech in V1's 6 minutes (72%), 4.3 in V2's 9 (48%), 4.2 in V3's 9 (47%) and 3.8 in V4's 7 (54%). V2 and V3 spend more time on silent work: typing names, wiring nodes, waiting for results. The build script prints these figures and names any shot whose narration runs faster than about 160 words per minute (none at present). If an edited video comes out shorter than planned, the shorter length is correct: silence is cut, not filled.
- **Listing videos.** Fab visitors skim a gallery. The first 12 seconds must show one feel off / feel on difference. T1 gives each of the four demos 10 to 12 seconds, the editor 13, one node 7, comfort 8 and the end card 5 (75 seconds). T2 keeps to what Lite contains and needs 40 seconds.
- **The Minutes column** of the video table prints in the manual as "about N minutes". After the edit, enter the final length rounded to whole minutes.

2 Recording setup

IN THIS CHAPTER

- 2.1 This machine
- 2.2 Software
- 2.3 Capture resolution and application scale
- 2.4 OBS profile, scene and sources
- 2.5 Microphone and room
- 2.6 Windows preparation
- 2.7 Unreal preparation
- 2.8 Pre-recording checklist

2.1 This machine

Item	Measured	Consequence
Monitors	Two, each 1920 x 1080, running at 144 Hz	Record monitor 1; keep OBS, this guide and notes on monitor 2
Graphics card	AMD Radeon RX 7900 XTX	Hardware encoder is AMD AMF: in OBS "AMD HW H.264 (AVC)" or "AMD HW H.265 (HEVC)". NVENC is NVIDIA only and does not apply here
Processor	AMD Ryzen 7 5700X, 8 cores	x264 software encoding at 1080p60 works but competes with Unreal for the processor; use the hardware encoder
Free space	D: 698 GB, B: 146 GB	Raw recordings go to D:/FeelKitVideo/ , outside the folder Synology Drive syncs
Installed	No OBS Studio, no DaVinci Resolve, no ffmpeg (the PATH names two ffmpeg folders that do not exist)	Install the software in the next section before the first session

Refresh rate. At 144 Hz, a 60 fps capture takes frames at uneven intervals ($144 / 60 = 2.4$ refreshes per frame), and motion judders. For recording sessions, set monitor 1 to 120 Hz (every captured

frame is exactly two refreshes) or 60 Hz: Windows **Settings > System > Display > Advanced display > Choose a refresh rate**. In Unreal, cap the frame rate with the console command `t.MaxFPS 60`.

2.2 Software

Program	Use	Where from	Cost
OBS Studio (current release)	Screen and audio recording	obsproject.com	Free
DaVinci Resolve (free version)	Editing, captions check, loudness	blackmagicdesign.com	Free
Microsoft PowerToys	Mouse Highlighter for clicks	Microsoft Store or GitHub	Free
ffmpeg (optional)	Only for the automated captures and loudness checks in the last chapter	ffmpeg.org builds	Free

The person recording downloads and installs these; the developer does not download or install programs.

2.3 Capture resolution and application scale

Three ways to capture. The recommended one is 2560 x 1440 through AMD Virtual Super Resolution: the 1080p monitor then offers higher resolutions in Windows, Unreal renders the editor with more pixels, and OBS records all of them. Text stays sharp after YouTube's compression, and zoom-ins in the edit stay sharp up to 1.33x.

- Turn it on in **AMD Software: Adrenalin Edition > Gaming > Display > Virtual Super Resolution**, then pick 2560 x 1440 for monitor 1 in Windows display settings. The physical screen shows the picture scaled down.
- Unreal's **Application Scale** keeps the interface at a readable size (table below). At 1.5 on a 2560 x 1440 desktop shown on a 1080p screen, the person recording sees the interface at 1.125 times its normal size.
- Verify while preparing: 2560 x 1440 appears in the Windows resolution list after VSR is on, and Unreal stays at 60 fps in the Third Person level at that resolution.

Capture resolution	Application Scale	Interface size on the recording	Sharp zoom in a 1080p export	Export for YouTube
1920 x 1080 (native)	1.25	Reference size	None (a zoom enlarges pixels; keep zooms at 1.25x or less)	1920 x 1080, 60 fps

Capture resolution	Application Scale	Interface size on the recording	Sharp zoom in a 1080p export	Export for YouTube
2560 x 1440 (VSR), recommended	1.5	90% of reference, slightly more room in panels	Up to 1.33x	2560 x 1440, 60 fps (YouTube streams 1440p at a higher bitrate)
3840 x 2160 (VSR)	2.25	90% of reference	Up to 2x	2560 x 1440 or 3840 x 2160, 60 fps

The listing videos are always exported at 1920 x 1080, the size Fab asks for; a 1440p recording scaled down gives a sharper 1080p file.

2.4 OBS profile, scene and sources

1. **Profile** > **New**, name **FeelKit 1440p** (or **FeelKit 1080p**). **Scene Collection** > **New**, name **FeelKit**.
2. Add a scene named **Unreal** with a **Display Capture** of monitor 1. Window Capture misses Unreal's context menus, dropdowns and tooltips, which are separate windows, so it is not suitable for tutorials.
3. Add **Application Audio Capture**, set to the Unreal Editor window, for the game and editor sound. In **Settings** > **Audio**, set **Desktop Audio** to **Disabled**, so Windows sounds and other programs are not recorded.
4. Add the microphone as **Audio Input Capture** and give it the filters in the microphone section.
5. **Edit** > **Advanced Audio Properties**: microphone on track 1 only, Unreal on track 2 only. Separate tracks let the edit lower the game under the narration and raise it for results.

Setting (where in OBS)	1920 x 1080 source	2560 x 1440 source	3840 x 2160 source
Base (Canvas) Resolution (Settings > Video)	1920 x 1080	2560 x 1440	3840 x 2160
Output (Scaled) Resolution	1920 x 1080	2560 x 1440	3840 x 2160
Common FPS Values	60	60	60
Output Mode (Settings > Output)	Advanced	Advanced	Advanced
Recording Format	Hybrid MP4 when offered, otherwise MKV (then File > Remux Recordings)	Same	Same
Video Encoder	AMD HW H.264 (AVC)	AMD HW H.265 (HEVC)	AMD HW H.265 (HEVC)
Rate Control	CQP	CQP	CQP
CQP value	18	18	20
Keyframe Interval	1 s	1 s	1 s

Setting (where in OBS)	1920 x 1080 source	2560 x 1440 source	3840 x 2160 source
Preset	Quality	Quality	Balanced if frames drop, otherwise Quality
B-Frames	0 (faster scrubbing in the editor)	0	0
Approximate data rate	40 to 80 Mbps, up to 0.6 GB per minute	60 to 110 Mbps	100 to 180 Mbps
Audio Track (Recording tab)	Tracks 1 and 2 ticked	Same	Same
Audio Encoder and bitrate (Audio tab)	AAC, 320 kbps per track	Same	Same
Color Format, Color Space, Color Range (Settings > Advanced)	NV12, Rec. 709, Limited	Same	Same
Sample Rate, Channels (Settings > Audio)	48 kHz, Stereo	Same	Same
Recording Path	D:/FeelKitVideo/Raw	Same	Same

Equivalents on another machine. NVIDIA: NVENC H.264 or HEVC, CQP 18, preset P5, Look-ahead off, Psycho Visual Tuning on, B-frames 0. No hardware encoder: x264, CRF 16, preset veryfast, profile high, keyframe interval 1 s.

Why constant quality and not a fixed bitrate: the recording is an intermediate file. Constant quality keeps still editor frames small and fast gameplay sharp, and the final bitrate is chosen at export. HEVC halves the file size at equal quality at 1440p and 4K. Verify on the first import that Resolve plays the HEVC files smoothly; if not, record H.264 at CQP 18 instead.

2.5 Microphone and room

- **Room.** The smallest furnished room available, curtains closed, door shut, no fan or air conditioning. Record the narration while Unreal is idle, so the computer's fans stay slow.
- **Position.** 10 to 15 cm from the mouth, aimed at the corner of the mouth rather than straight at it, with a pop filter. The microphone model is not recorded in the project; a condenser microphone picks up more of the room than a dynamic one, so the room matters more with it.
- **Windows.** **Settings > System > Sound > the microphone > Audio enhancements** off. In **Control Panel > Sound > Recording > the microphone > Properties > Advanced**, choose 1 channel, 24 bit, 48000 Hz.
- **Level.** Normal speech peaks around -12 dBFS and loud words stay below -6 dBFS on the OBS mixer: mostly in the yellow section, never red.
- **Room tone.** At the start of each session, record 10 seconds of silence with the microphone open. The edit uses it to fill gaps and to measure the noise floor.
- **Monitoring.** Closed headphones. Speakers leak the game's sound into the microphone.

Filters on the microphone source (right-click the source > **Filters**), in this order:

Order	Filter	Settings	Purpose
1	Noise Suppression	Method RNNoise	Removes steady fan noise and hum
2	Noise Gate	Close Threshold -45 dB, Open Threshold -38 dB, Attack 25 ms, Hold 200 ms, Release 150 ms	Silences the room between phrases. Set Close about 6 dB above the room tone shown on the meter; if word endings are cut, lower both thresholds by 5 dB
3	Compressor	Ratio 3:1, Threshold -20 dB, Attack 6 ms, Release 60 ms, Output Gain +3 dB, no sidechain	Brings loud and soft words closer together
4	Limiter	Threshold -3 dB, Release 60 ms	Stops clipping on sudden loud words

OBS writes the filtered sound into the recording; the unfiltered sound is not kept. When the edit is to do its own noise reduction, turn filters 1 and 2 off and keep 3 and 4. The final loudness is set in the edit, not in OBS.

2.6 Windows preparation

- **Do not disturb** on (**Settings** > **System** > **Notifications**). Close Discord, Steam, browsers and chat programs.
- **Pause Synology Drive** for the session. It syncs `B:/NewUE5Project`, and it restored deleted folders once before.
- **Taskbar** hidden on monitor 1 (**Settings** > **Personalization** > **Taskbar** > **Taskbar behaviors** > **Automatically hide the taskbar**); desktop icons hidden; a plain dark gray wallpaper.
- **Display scale** 100% on monitor 1 (**Settings** > **System** > **Display** > **Scale**), so Unreal's Application Scale alone sets the interface size.
- **Pointer. Settings** > **Accessibility** > **Mouse pointer and touch**: white style, size 2. PowerToys **Mouse Highlighter**: primary button color 2E8B57 at 60% opacity, secondary button color 1F6FB2, radius 20 px, fade delay 400 ms, fade duration 250 ms, turned on with Win+Shift+H. Leave **Find My Mouse** off.
- **Keyboard layout** English (US), so the keys named in the narration match the keyboard.
- **Power**. Power mode **Best performance**; screen saver and sleep off.

2.7 Unreal preparation

The recording project. A new project made for the videos, never GameFeelDev (it holds test content and tools) or FeelDemoFP.

- **Engine:** UE 5.8, the newest version the listing supports and the one a new buyer most likely has. FeelKit's own windows are the same on 5.6, 5.7 and 5.8. Verify the Third Person template's names on 5.8 while preparing (`BP_ThirdPersonCharacter`, `Lvl_ThirdPerson`, the animations `MM_Jump` and `MM_Land`).
- **Template:** **Games > Third Person, Blueprint, Desktop**, quality **Maximum**, **Starter Content** off. Name `FeelTutorial`, location `D:/FeelKitVideo/Projects`.
- **FeelKit installed the buyer's way.** After the listings are live: from Fab through the Epic Games Launcher (V1 shows this). Before that: the release package for 5.8 from `Tools/Run/package_plugin.ps1`, placed in the engine the way `Tools/Run/check_engine_plugin.ps1` does; the developer prepares it. Remove that copy before installing from Fab.
- **Snapshots.** With the editor closed, copy the project folder (without `DerivedDataCache` and `Saved`) after each video: `FeelTutorial_after_V1`, `_after_V2`, and so on. A retake then starts from the exact state the script assumes.

Editor settings (Editor Preferences unless stated):

- **General > Appearance > User Interface > Application Scale:** 1.25 at 1080p, 1.5 at 1440p, 2.25 at 4K.
- **General > Performance:** **Use Less CPU when in Background** off; **Show Frame Rate and Memory** off.
- **General > Loading & Saving:** automatic saving off for the recording sessions (its notification would appear in shots). Save by hand after each video.
- **Level Editor > Play: Game Gets Mouse Control** on, so a click into the viewport is not needed. Shift+F1 frees the mouse during Play In Editor.
- **Layout: Window > Load Layout > Default Editor Layout.** Close any window that does not appear in the scripts. Dock the Content Browser at the bottom (**Dock in Layout** in the Content Browser drawer) so it stays visible. Then **Window > Save Layout > Save Layout As**, name `FeelKit Video`, and load it at the start of each session.
- **Frame rate:** `t.MaxFPS 60` in the Output Log's command line at the start of each session.
- **Motion blur:** leave it as the template ships it. The manual turned it off only for one still picture (the hit picture in 4.3), because blur in a still hides the effect; a video shows the game as players see it.
- **Sound:** Unreal at full volume, Windows volume fixed for the whole production (for example 50%).
- **Controller:** an Xbox-compatible (XInput) controller for the rumble shots. Vibration is invisible on screen: the narration names it, and V4 shows it in the Debugger. PlayStation controllers vibrate on Windows only through Steam Input.

2.8 Pre-recording checklist

1. Monitor 1 at 120 Hz or 60 Hz, at the capture resolution; VSR on if recording 1440p or 4K.

2. Synology Drive paused; Do not disturb on; Discord, Steam, browsers and chat closed.
3. Taskbar hidden, desktop icons hidden, plain wallpaper.
4. PowerToys Mouse Highlighter on (Win+Shift+H), pointer size 2.
5. OBS: profile and scene collection loaded, recording path on D:, free space above 50 GB.
6. OBS mixer: microphone peaks at -12 dBFS on a test sentence; Unreal on track 2; Desktop Audio disabled.
7. Unreal: project snapshot for this video opened; layout `FeelKit Video` loaded; `t.MaxFPS 60` entered.
8. Application Scale set for the capture resolution; auto save off.
9. The starting state in the video's setup table checked item by item.
10. Controller connected (V1, V2, V4), headphones on.
11. A 10-second test recording played back in Resolve: picture sharp, both audio tracks present, no dropped frames (**View > Stats** in OBS shows 0 skipped frames).
12. Room tone recorded (10 seconds).
13. Script open on monitor 2 at the shot about to be recorded; take log ready (next chapter).

3 Narration and screen style

IN THIS CHAPTER

- 3.1 Voice and wording
- 3.2 Pointer and keyboard
- 3.3 Takes and the take log

3.1 Voice and wording

- The narration is recorded in the person's own voice. It uses the manual's tone: precise, formal and plain, in complete sentences, present tense, and "you" for the viewer.
- **Pace:** 140 words per minute. Pause about half a second after naming a control and before clicking it, and a full second after a result appears.
- **Labels:** say each label exactly as the editor shows it. Menu paths are spoken as a list: "Tools, FeelKit Recipe Browser".
- **Asset names:** read the prefix as letters: "F R Impact HeavyHit" for `FR_Impact_HeavyHit`, "B P Third Person Character" for `BP_ThirdPersonCharacter`.
- **Terms:** "hitstop" is one word. "FOV Kick" is read as it is written. Say "Play In Editor" in full.
- **Numbers:** read as displayed, with the unit: "zero point zero eight seconds".
- **Claims:** describe only what is on screen. No product comparisons, no superlatives and no sales adjectives; the words the manual's lint rejects are rejected here too.
- **Spelling** on screen is American (color, behavior, center). Unreal's own labels keep their spelling.

3.2 Pointer and keyboard

- Move the pointer at about half the usual speed, in straight lines. Keep it still while a result plays, and take the hand off the mouse during result shots.

- Rest on a control for half a second before clicking it, so the viewer can find it on their own screen. Tooltips appear when resting longer; wait for one only when the script asks for it.
- Say the key before pressing it ("Press Tab"). The edit adds a key caption at the moment of the press.
- A mistake means a retake of the shot. A cut inside a pointer movement makes the pointer jump, which reads as edited.

3.3 Takes and the take log

Method: picture first, voice second. Record each shot (or a group of shots marked in the Notes column) as its own clip, with 2 seconds of stillness before and after. Then record the narration shot by shot while watching the clip, two takes per shot. The edit trims the picture to the voice. This keeps the pointer slow and deliberate and lets a sentence be redone without redoing the actions. Recording the voice live while acting also works for V1; the timings in the scripts assume the edit either way.

OBS cannot name files by shot. Keep a take log on paper or in a spreadsheet that is not saved in the project folder, with the columns: clip file name (OBS time stamp), video, shot, take, keep (yes or no), note. Rename kept clips to `V2_S07_T2.mp4` (video, shot, take) at the end of the session.

4 Video 1: Install FeelKit and play your first recipe

IN THIS CHAPTER

4.1 Setup

4.2 Script

4.1 Setup

The flow follows the manual's chapter 4 (written and approved) and the install steps planned for 3.2 and 3.4. The earlier plan in section 2 of this document picked the recipe in the Recipe Browser with **Use**; the approved chapter 4 uses the Content Browser and **Copy to Project**, which works in Lite and Pro, so the script follows chapter 4.

Item	State
Starting state	UE 5.8 installed; FeelKit not yet installed (shot 3) or installed from the release package (all other shots); Epic Games Launcher open and signed in; no project
Ending state	Project <code>FeelTutorial</code> with <code>/Game/HeavyHit</code> (copy of <code>FR_Impact_HeavyHit</code>), the 1 key in <code>BP_ThirdPersonCharacter</code> playing it, a Feel Switch in <code>Lvl_ThirdPerson</code> ; snapshot <code>FeelTutorial_after_V1</code>
Keys used	1, Tab, O, Space, Esc
Edition	Lite and Pro. Verify while recording: <code>FR_Impact_HeavyHit</code> is in the Lite library once the Lite recipe list is decided. If it is not, use the Lite recipe chosen then; the steps do not change
Record when	Shots 1 and 4 to 18 before submission; shot 3 after the listings are live, since the Fab Library only lists a product once it is published
YouTube chapters	0:00 The result; 0:27 Installing from Fab; 0:50 A Third Person project; 1:05 Enabling the plugin; 1:30 FeelKit content; 1:55 A library recipe; 2:50 Previewing; 3:55 Playing it from a Blueprint; 4:55 In the game; 5:15 The Feel Switch; 6:00 Next

4.2 Script

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
1	0:00-0:15	Play In Editor in Lv1_ThirdPerson , badge FEEL: ON	Press 1 twice, 2 s apart. Press Tab (FEEL: OFF), press 1. Press Tab (FEEL: ON), press 1	This is FeelKit on Unreal's Third Person template. The 1 key plays a recipe: a short freeze, a camera kick, a flash, a sound and a controller rumble. With FeelKit switched off, the same key does nothing.	Title, lower left: Install FeelKit and play your first recipe	Record last, from the finished state. Follow the feel off / feel on rules in the editing chapter
2	0:15-0:27	Title card over a still of the recipe editor	None	In the next six minutes: install FeelKit, copy a recipe from its library, preview it, play it from a Blueprint, and compare the game with the feel on and off.	Title card, 12 s	Title card design in the editing chapter
3	0:27-0:50	Epic Games Launcher	Unreal Engine > Library ; scroll to Fab Library ; search FeelKit; Install to Engine ; choose 5.8; Install	Like every code plugin from Fab, FeelKit installs into the engine. In the Epic Games Launcher, open Unreal Engine, then Library. In the Fab Library, find FeelKit and click Install to Engine. Choose your engine version and click Install.	Callout on Install to Engine	Verify while recording: launcher labels at the first real install. Cut the download wait. Hide the account name if it shows (blur in the edit)
4	0:50-1:05	Unreal Project Browser	Games > Third Person > Blueprint ; Starter Content off; name FeelTutorial ; Create	Create a project from the Third Person template. A Blueprint project is enough: nothing in this video needs C++.	Lower third: Games > Third Person > Blueprint	Cut the project creation and shader compile. The Recent Projects list must show no other projects
5	1:05-1:30	Editor, then the Plugins window	Edit > Plugins ; type FeelKit; tick Enabled ; Restart Now	Open Edit, Plugins, and search for FeelKit. Tick Enabled and restart the editor. FeelKit also turns on the two engine plugins it uses, Niagara and Enhanced Input, so nothing else needs enabling.	Callout on Enabled; version 1.0.0 readable	Hold 1 s on the version. Verify while recording whether Unreal asks to confirm the dependent plugins; if it does, click Yes and keep it in the shot. Cut the restart

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
6	1:30-1:55	Content Browser	Content Browser Settings > Show Plugin Content ; in the source tree All > Plugins > FeelKit Content ; open Library > Impact	FeelKit's content is part of the plugin, and the Content Browser hides plugin content by default. Open Settings in the Content Browser and turn on Show Plugin Content. FeelKit Content now appears under Plugins. Its Library folder holds ready-made recipes, grouped by feeling. Open Impact.	Lower third: Settings > Show Plugin Content	Zoom 1.33x on the Settings menu (1440p source)
7	1:55-2:15	Impact folder	Rest on FR_Impact_HeavyHit until the tooltip shows; double-click it	A recipe is a short timeline of feedback effects. F R Impact HeavyHit is a heavy blow: a brief freeze, a camera shake and punch, a scale punch on the character, a flash, a sound and a rumble. Double-click it to open the recipe editor.	None	The tooltip shows Description, Tracks and Length; hold it 2 s
8	2:15-2:40	Recipe editor, read-only banner	Point at the banner; Copy to Project ; in the save dialog keep HeavyHit in the Content folder; Save	Library recipes are read-only, so that an update of FeelKit can replace them without touching your project. The banner says so. Click Copy to Project, keep the name HeavyHit, and save it in your project. The copy opens, and you can change it freely.	Spotlight on the banner, as in the manual's figure	The save dialog must not show a Windows user path
9	2:40-2:50	Still: Tools menu and the Recipe Browser	None	In FeelKit Pro, Tools, FeelKit Recipe Browser shows the same library with filters, and plays each recipe when you rest the pointer on it.	PRO Label, top right	A still is enough; recorded with Pro installed
10	2:50-3:20	Recipe editor with HeavyHit	Play in the Timeline toolbar; then Loop ; let it loop three times	The recipe editor plays the recipe on its own camera and mesh, with the same code as the game. Click Play in the Timeline toolbar, or press Space. Click Loop to repeat it while you watch and listen.	Callouts on Play and Loop	Loop gives two flashes per second; stay under three flashes in any second of the video
11	3:20-3:40	Timeline	Loop off; drag the red playhead slowly from 0 to 0.5 s and back	Drag the playhead along the ruler to step through the recipe. Scrubbing backward shows exactly the frames that playing forward shows, because each effect is computed from the time alone.	None	Slow drag, about 4 s each way

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
12	3:40-3:55	Timeline, two rows	Rest on the No preview label of Global Hitstop, then of Force Feedback Curve	Two tracks say No preview: Global Hitstop, which slows the game's time, and Force Feedback Curve, which drives the controller. Both act only in a running game.	Callout on each label	
13	3:55-4:25	Content Browser, then the Blueprint editor	Open BP_ThirdPersonCharacter (ThirdPerson > Blueprints); Event Graph ; right-click an empty area; type 1; Input > Keyboard Events > 1	Now play it from a Blueprint. Open the character Blueprint, B P Third Person Character, and its Event Graph. Right-click an empty area, type 1, and choose the keyboard event for the 1 key.	Lower third: Input > Keyboard Events > 1	Verify the Blueprint's folder on the 5.8 template
14	4:25-4:55	Event Graph	Drag from Pressed , type Play Feel, choose Play Feel (Feel); set Recipe to HeavyHit ; drag from Target , choose Make Feel Target from Actor ; right-click, Get a reference to self , connect to Actor ; Compile ; Save	Drag from Pressed and add Play Feel, from the Feel category. Set Recipe to HeavyHit. Drag from Target, choose Make Feel Target from Actor, and connect a reference to self. The target decides where each effect goes. Compile and save.	Zoom 1.33x on the node while the pins are set	Same graph as the manual's figure 4.4
15	4:55-5:15	Play In Editor	Play ; press 1 twice, 3 s apart	Press Play, then 1. The camera kicks and shakes, the screen flashes, the character's mesh punches in scale, time freezes for a tenth of a second, and a connected controller rumbles. FeelKit moves and scales the mesh, never the collision capsule.	Key caption: 1. Caption at the second press: Controller rumbles	Keep the character still and the camera level
16	5:15-5:40	Editor, Place Actors panel	Esc; Window > Place Actors ; search Feel Switch; drag it into the level; Play	To compare, add a Feel Switch. Open Window, Place Actors, search for Feel Switch and drag it into the level. Press Play. A start card explains the switch, and a badge shows FEEL: ON.	Lower third: Window > Place Actors	Wait for the start card before speaking its sentence
17	5:40-6:00	Play In Editor	Press 1; Tab; 1; Tab; 1; then O (menu opens), O (menu closes)	Press Tab to turn FeelKit off, and press 1 again: the same moment, without FeelKit. Tab turns it back on. O opens the comfort settings for players, which the comfort video covers.	Key captions: Tab, O	The Feel Switch opens the menu with O by default
18	6:00-6:08	End card	None	Chapter 4 of the manual has these steps in writing. The next video builds a recipe on the timeline.	End card: next video and the manual link	Hold 8 s if a YouTube end screen is added

5 Video 2: Build a recipe on the timeline

IN THIS CHAPTER

5.1 Setup

5.2 Script

5.1 Setup

Item	State
Starting state	Snapshot <code>FeelTutorial_after_V1</code> ; FeelKit Pro installed; recipe editor windows closed; controller connected
Ending state	<code>/Game/Hit</code> with six tracks (Global Hitstop, Camera Punch, Procedural Shake, Screen Flash, Play Sound with <code>S_FK_Hit_Heavy</code> , Force Feedback Curve) and the parameter Damage mapped on the shake and the punch; snapshot <code>FeelTutorial_after_V2</code>
Keys used	Space, F, Shift, Ctrl+wheel, Ctrl+C, Ctrl+V, Ctrl+S, Delete, Shift+F1
Edition	Lite and Pro. Shot 11 (parameters) is Pro. The waveform on the Play Sound track is Pro; in Lite the row shows no waveform
YouTube chapters	0:00 The result; 0:15 A new recipe; 0:40 Adding tracks; 2:35 Moving and resizing; 3:20 Intensity curves; 4:05 Step settings; 4:45 Previewing; 5:15 Parameters (Pro); 6:05 Comfort preview; 6:40 Play in PIE; 7:40 Validation; 8:20 Copy and paste; 8:40 Next

5.2 Script

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
1	0:00-0:15	Finished Hit looping in the recipe editor, then one hit in Play In Editor	Loop on for 6 s; cut to the game, press 2	This video builds a hit recipe from an empty timeline: a freeze, a camera punch, a shake, a flash, a sound and a rumble, tuned in the preview without pressing Play.	Title, lower left: Build a recipe on the timeline	Record last. Key 2 is wired in V3; for this shot, use the Play in PIE button instead
2	0:15-0:40	Content Browser, then the recipe editor	Right-click in the Content folder > FeelKit > Feel Recipe ; name Hit ; double-click it	Right-click in the Content Browser and choose FeelKit, Feel Recipe. Name it Hit and open it. The recipe editor has four areas: Preview, Timeline, Details and Intensity.	Numbered markers 1 to 4 on the areas, 4 s	Markers match the manual's figure style
3	0:40-1:05	Timeline	+ Track > Global Hitstop ; drag the track's right edge to about 0.08 s	Click plus Track and choose Global Hitstop. A hitstop freezes the game for a moment, and it works best when it is short. Drag the end of the track to about zero point zero eight seconds. The edge snaps to frames.	Callout on + Track	Snap on (toolbar)
4	1:05-1:50	Timeline	+ Track > Camera Punch ; + Track > Procedural Shake ; + Track > Screen Flash	Add Camera Punch, Procedural Shake and Screen Flash the same way. Each track plays one step. Each step belongs to a channel, which colors the row and decides which comfort setting scales it.	None	One continuous take; rest the pointer on each new row for 1 s
5	1:50-2:15	Timeline and Details	+ Track > Play Sound ; in Details set Sound to S_FK_Hit_Heavy	Add Play Sound, and in Details set Sound to S F K Hit Heavy, one of the sample sounds that come with FeelKit. In FeelKit Pro, the row shows the sound's waveform.	PRO Label while the waveform is visible	Type Hit_Heavy in the asset picker's search
6	2:15-2:35	Timeline	+ Track > Force Feedback Curve	Last, add Force Feedback Curve for the controller. Like the hitstop, it says No preview, because it acts only in a running game.	None	
7	2:35-3:20	Timeline	Resize Camera Punch to 0.4 s, Procedural Shake to 0.3 s, Screen Flash to 0.12 s; drag one track while holding Shift; press F; Ctrl+wheel to zoom in and out	Drag a track to move it, and drag its ends to change its length. Snap keeps edges on frame steps; hold Shift to place them freely. Press F to fit the whole recipe into view, and use Ctrl and the mouse wheel to zoom.	Key captions: Shift, F, Ctrl + wheel	

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
8	3:20-4:05	Timeline, curve lane of the shake	Select Procedural Shake; double-click the lane at the start and near the end; drag the last key down to 0; right-click a key > Smooth	Select the shake track, and its Intensity lane opens below it. With no keys, the track plays at full strength. Double-click to add a key at the start and one near the end, then drag the last key down to zero, so the shake fades out. Right-click a key to choose Constant, Linear or Smooth.	Callout on the lane	Zoom 1.33x on the lane while adding keys
9	4:05-4:45	Details of Camera Punch	Select Camera Punch; in Details set Shape to Kick ; raise Rotation Punch pitch; click Play	Select Camera Punch, and Details shows the step's settings. The default shape, Spring, overshoots and settles. Kick pushes fast and returns smoothly. Switch Shape to Kick, and raise the Rotation Punch pitch for a stronger kick.	Zoom 1.25x on the Details category	Use the property names exactly as Details shows them
10	4:45-5:15	Preview and Timeline	Loop on; scrub the playhead to the frame of the strongest punch; rest on the No preview labels	Loop the preview while you tune. Scrub the playhead to find the exact frame of the punch. The hitstop and the rumble are marked No preview; Play in PIE, later in this video, shows them.	None	Two flashes per second while looping; acceptable
11	5:15-6:05	Details (recipe), Timeline slider	Click an empty area of the timeline so no track is selected; in Details Parameters add Damage , Min 0, Max 100, Default 50; select the shake, Parameter Mappings add Damage ; same on Camera Punch; drag the Damage slider under Preview parameters from 0 to 100 while looping	In FeelKit Pro, one recipe can serve light and heavy hits. With no track selected, Details shows the recipe. Add a parameter named Damage, from 0 to 100. On the shake and the punch, add a Parameter Mapping to Damage. A Damage slider now appears under Preview parameters: drag it, and the preview follows. In the game, Play Feel with Context passes the value.	PRO Label, top right	Verify while recording: an empty-area click clears the track selection. The slider values are not saved
12	6:05-6:40	Timeline toolbar, Intensity tab	Comfort > Reduced Motion ; play; Comfort > Neutral ; open the Intensity tab	The Comfort dropdown previews the recipe as a player with other settings would see it. Reduced Motion plays camera shake and camera motion at a quarter of their strength, and each row shows the scale it received. The Intensity tab plots each channel's strength across the recipe.	Callout on the Comfort x0.25 labels	

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
13	6:40-7:40	Level editor and recipe editor side by side	Play ; Shift+F1; in the recipe editor click Play in PIE ; change Screen Flash color; Play in PIE again	Play in PIE shows what the preview cannot. Start Play In Editor, then click Play in PIE in the recipe editor. The recipe plays on the player's character, with the hitstop and the rumble. Change a value, click Play in PIE again, and the change is already there, without restarting the game.	Key caption: Shift + F1. Caption: Controller rumbles	Arrange the recipe editor on the right half of monitor 1 before recording
14	7:40-8:20	Timeline, Message Log	Stop play; + Track > Play Sound without a sound; Ctrl+S; read the message; select the track; Delete; Ctrl+S	Saving checks the recipe. A Play Sound track without a sound gives a message that names the track and the problem: Track 7, Play Sound has no sound. Delete the track, or set a sound, and save again.	Zoom 1.33x on the message	Verify while recording where the message appears (Message Log or notification)
15	8:20-8:40	Two recipe editors	Select the Screen Flash track of Hit ; Ctrl+C; open HeavyHit ; move the playhead; Ctrl+V	Tracks copy between recipes. Ctrl+C copies the selected track, and Ctrl+V in another recipe pastes it at the playhead.	Key captions: Ctrl + C, Ctrl + V	Undo the paste (Ctrl+Z) after the shot, so HeavyHit keeps its seven tracks
16	8:40-9:00	End card	None	Chapter 5 of the manual describes every part of the recipe editor. The next video plays recipes from gameplay: from Blueprints, events, animations and components.	End card	

6 Video 3: Trigger recipes from your game

IN THIS CHAPTER

6.1 Setup

6.2 Script

6.1 Setup

Item	State
Starting state	Snapshot FeelTutorial_after_V2 . Prepared off camera with Copy to Project: /Game/ChargeUp (from FR_Power_ChargeUp , parameter Charge, sustain from 0.35 to 0.95 s, Release Parameter Charge at 1, On Full Release FR_Power_ChargedRelease) and /Game/Land (from FR_Weight_Land , parameter FallSpeed, 0 to 1600, default 600)
Ending state	Keys 2 to 5 wired in BP_ThirdPersonCharacter ; Feel Map /Game/FM_Game in the project settings; a Feel Trigger on the character; snapshot FeelTutorial_after_V3
Keys used	1 to 5, Space (jump), Esc, Shift+F1
Edition	PRO Play Feel and Stop Feel are also in Lite; the video is published as a Pro tutorial with a PRO label in the title card
YouTube chapters	0:00 Six ways to play a recipe; 0:15 Play Feel with Context; 1:15 Handles, release and parameters; 2:30 Events and Feel Maps; 3:45 Animation notifies; 5:05 The Feel Trigger component; 6:40 The Debugger; 7:50 More ways; 8:30 Next

6.2 Script

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
1	0:00-0:15	Play In Editor montage	Press 2; hold and release 3; jump from a ledge	Gameplay decides when something happens, and FeelKit decides how it feels. This video connects recipes to gameplay in six ways, from a single node to a component that needs no Blueprint at all.	PRO Title: Trigger recipes from your game, with the label	Record last
2	0:15-0:55	Event Graph of the character Blueprint	Add keyboard event 2; Play Feel with Context : Recipe Hit , Target from Make Feel Target from Actor with self; drag from Context > Make FeelPlayContext ; under Parameters add Damage = 90	Play Feel with Context plays a recipe with extra information. Connect Make FeelPlayContext to Context, and under Parameters, add Damage with the value 90. Instigator and Location serve tracks that act on a second actor or need a position; this recipe uses neither. With an empty context, the node behaves exactly like Play Feel.	Lower third: Feel > Play Feel with Context	Verify while recording: if the Parameters pin takes no typed values, drag from it and add a Make Map node with the key Damage and the value 90
3	0:55-1:15	Play In Editor	Compile; Play; press 2; Esc; set the value to 20; Play; press 2	Press 2: the hit plays at a damage of 90. With the value set to 20, the same recipe plays as a light hit.	Captions: Damage 90, Damage 20	Keep both hits in the same camera position
4	1:15-2:05	Event Graph	Key 3 Pressed > Play Feel (ChargeUp) ; right-click Return Value > Promote to Variable , name ChargeHandle ; Released > Release Feel (Handle ChargeHandle); key 4 > Set Feel Parameter (Handle ChargeHandle , open the name dropdown, choose Charge, Value 1)	Play Feel returns a handle: a reference to that one play. Promote it to a variable. ChargeUp, a copy of the library's charge recipe, is sustained: it loops its middle part until released. When the key is released, Release Feel ends the loop. Set Feel Parameter changes a value while the recipe plays. Its dropdown lists the parameters of the recipe the handle came from, here Charge.	Callout on the dropdown	Stop Feel is mentioned in shot 5
5	2:05-2:30	Play In Editor	Play; hold 3 for 3 s; press 4 while holding 3; release 3; hold 3 again for 2 s; release 3 without pressing 4	Hold 3: the charge builds and holds. Press 4: Charge reaches 1, and the recipe releases itself with the burst set in its On Full Release. Hold again and let go early: the hum fades, without the burst. Stop Feel would end the play at once.	Key captions: 3 held, 4, 3 released	Verify while recording: the burst plays on 4 without releasing 3

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
6	2:30-3:30	Content Browser, Feel Map editor, Project Settings, Event Graph	Right-click > FeelKit > Feel Map , name FM_Game ; open; Entries add: Event Feel.Event.Hit.Landed , Recipe Hit ; save; Edit > Project Settings > Plugins > FeelKit > Feel Maps add FM_Game ; key 5 > Send Feel Event (Event Feel.Event.Hit.Landed , Target self)	Events keep recipe choices out of gameplay code. Create a Feel Map: right-click, FeelKit, Feel Map. Add a row with the event Feel Event Hit Landed and the recipe Hit. In Project Settings, Plugins, FeelKit, add the map under Feel Maps. Send Feel Event with that event now plays whatever the map assigns. Rows with Required Tags choose variants, such as a critical hit, without a branch in Blueprint.	Lower third: Project Settings > Plugins > FeelKit > Feel Maps	One continuous take is not needed; cut between the four windows
7	3:30-3:45	Play In Editor	Play; press 5	Press 5, and the event plays Hit. When no row matches an event, the Output Log says so, once per event.	Key caption: 5	
8	3:45-4:45	Animation editor, MM_Land	Open MM_Land ; right-click the Notifies track > Add Notify > Play Feel ; in Details set Recipe to Land ; scrub over the notify	Animation notifies play recipes at an exact frame. Open an animation, here the template's landing animation. Right-click the Notifies track, choose Add Notify, then Play Feel, and set Recipe to Land. Notifies also play in the animation editor, so actor effects, such as the squash on landing, can be timed while you scrub.	Lower third: Notifies > Add Notify > Play Feel	Verify while recording: the MM_Land path on 5.8, and which of Land's effects show in the animation preview (actor effects only)
9	4:45-5:05	Notify menu	Open Add Notify and Add Notify State menus without choosing	Three more notifies exist: Send Feel Event, Set Feel Value, which passes a value such as the weight of a swing to later recipes, and Play Feel Window, a notify state that plays a recipe for the length of a window.	None	
10	5:05-6:10	Animation editor, then the Blueprint editor	Delete the Play Feel notify from MM_Land ; save; in BP_ThirdPersonCharacter Add > Feel Trigger ; in Details Triggers add: Event Landed, Recipe Land , Value Parameter FallSpeed; Compile; Save	The notify plays every landing at the same strength. The Feel Trigger component can pass the real landing speed. Remove the notify, and add a Feel Trigger to the character. Add an entry: Event Landed, Recipe Land, Value Parameter FallSpeed. The landing speed goes into the recipe's FallSpeed parameter, and no Blueprint wiring is needed.	Lower third: Add > Feel Trigger	

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
11	6:10-6:40	Play In Editor	Play; a small jump on flat ground; then run off the highest ledge	A short hop lands softly. A long fall lands hard, from the same entry. The Feel Trigger also answers damage, hits, overlaps, jumps, air jumps and launches.	None	Keep the camera behind the character for both landings
12	6:40-7:50	Debugger tab beside the game	Tools > FeelKit Debugger ; play 2, 3 and 5 in the game; stop play; in Recent plays double-click the Hit play	Tools, FeelKit Debugger lists what plays right now, the accumulators, each player's comfort settings and the controller's vibration. Every play that ends is kept under Recent plays. Double-click one, and the recipe editor opens that exact play, with its parameters, to scrub it frame by frame.	Callout on Recent plays	Recent plays are kept after play stops, not in Shipping builds
13	7:50-8:30	Stills: Feel Input details, a Gameplay Cue Blueprint, a line of C++	None	Chapter 6 of the manual covers three more ways: the Feel Input component for Enhanced Input actions, Gameplay Cue notifies from the GAS add-on, and the Feel subsystem for C++.	Captions naming each still	Stills from the manual's figures when available
14	8:30-9:00	End card	None	The next video covers comfort: the settings that let each player choose how much shake, flash and rumble they get.	End card	

7 Video 4: Comfort settings for your players

IN THIS CHAPTER

7.1 Setup

7.2 Script

7.1 Setup

Item	State
Starting state	Snapshot FeelTutorial_after_V3 ; comfort save slots deleted (the project's Saved/SaveGames folder emptied with the editor closed), so the player starts with the default settings
Ending state	Keys 7, 8 and 9 wired to comfort nodes; the Screen Flash track of HeavyHit marked Essential with a Vignette Pulse substitute; snapshot FeelTutorial_after_V4
Keys used	1, 7, 8, 9, O, Ctrl+D, Esc
Edition	Lite and Pro. Shots 10 and 11 are Pro
YouTube chapters	0:00 Comfort in FeelKit; 0:15 Groups and presets; 0:50 Comfort in the preview; 1:25 Changing settings from Blueprint; 2:30 Saved settings; 3:00 Essential tracks; 4:00 The flash limiter; 4:40 The comfort menu; 5:30 Engine shakes and vibration; 6:05 The Debugger (Pro); 6:35 The Comfort Audit (Pro); 6:55 Next

Found while writing this script: the recipe editor shows "Substitute plays (comfort)" only when the preview's Flashes scale is 0, and none of the presets as shipped sets it to 0 (Reduced Flashing uses 0.2). The code plays a substitute only at a scale of 0 ([FeelEvaluator.cpp](#)). Shot 6 therefore shows the substitute in the game, after setting Flashes to 0 from Blueprint. The planned screenshot S08-04 needs the same change.

7.2 Script

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
1	0:00-0:15	Play In Editor with the comfort menu open	O; move Camera shake from 100% to 25% with the arrow keys; Try it	Players differ in how much shake, flash and rumble they can take. FeelKit gives every player comfort settings that scale its effects, and also the game's own camera shakes and controller vibration.	Title: Comfort settings for your players	Record last
2	0:15-0:50	Project Settings	Edit > Project Settings > Plugins > FeelKit ; expand Default Comfort Scales and the three presets	Comfort settings belong to each player: seven scales from 0 to 1. Master scales everything; the six groups are Camera Shake, Camera Motion, Flashes, Hitstop and Slow-mo, Screen Distortion and Haptics. A track's channel decides its group. New players start with the Default Comfort Scales, and the presets Reduced Motion, Reduced Flashing and No Haptics are edited here too.	Lower third: Project Settings > Plugins > FeelKit	Slow scroll; zoom 1.25x on the scales
3	0:50-1:25	Recipe editor with HeavyHit	Comfort > Reduced Motion ; Play; Comfort > No Haptics ; Comfort > Neutral	The recipe editor previews these settings. With Reduced Motion, the shake and the punch play at a quarter of their strength, and each row shows the scale it received. No Haptics removes the rumble. Neutral plays everything at full strength.	Callout on the Comfort x0.25 labels	
4	1:25-2:30	Event Graph of the character Blueprint	Key 7: Get Player Controller > Get Feel Comfort > Apply Comfort Preset (Reduced Motion). Key 8: Set Comfort Group Scale (Group Flashes, Scale 0). Key 9: Apply Comfort Preset (Default). Compile	In a game, the settings change through the player's comfort. Get Feel Comfort takes a player controller. On key 7, apply the preset Reduced Motion. On key 8, set the Flashes group to 0. On key 9, apply Default again. A real game connects these nodes to its options menu, or uses the comfort menu shown later in this video.	Lower third: Feel > Comfort	
5	2:30-3:00	Play In Editor	Play; 1; 7; 1; Esc; Play; 1; 9	Press 1 for the full effect, then 7 and 1 again: less shake and a smaller kick. Stop the game and start it again: the setting is still there. FeelKit saves each player's settings automatically, or hands them to your own save system.	Key captions	Same camera for all three presses

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
6	3:00-4:00	Recipe editor, then Play In Editor	In HeavyHit select Screen Flash ; Details > Comfort: Essential on, Substitute Step Vignette Pulse; save; Play; 1; 8; 1	Some effects carry information, such as a flash that signals damage. Mark such a track Essential. A player who turns Flashes to 0 then gets the Substitute Step instead, here a Vignette Pulse, so the signal stays without the flash. Without a substitute, the track keeps a minimum strength, its Essential Floor. On camera shake and camera motion, a player's 0 always wins.	Captions: Flashes 1, Flashes 0	See the note above the script
7	4:00-4:40	Recipe editor	Select Screen Flash ; Ctrl+D; drag the copy to 0.25 s; Comfort > Reduced Flashing ; Play; rest on the label of the second flash	The flash limiter counts flashes per second. Reduced Flashing allows one flash per second and suppresses the rest; the second flash's row says so. With the default settings, flashes beyond three per second are softened. The limiter is a helper, not a photosensitivity certification.	Callout on the label	Verify while recording that the label reads Flash suppressed by the flash limiter. Do not loop: two flashes 0.25 s apart. Delete the copy afterward
8	4:40-5:30	Play In Editor, comfort menu	Play; O; move through the rows with the arrow keys; click Reduced motion; Try it; Close ; then a still of /FeelKit/UI/WBP_FeelComfort Menu in the Content Browser with its right-click menu	Players change these settings in the comfort menu. The Feel Switch opens it with O, and in your own game the node Show Feel Comfort Menu opens it. It has Master and the six groups, the presets, camera roll, a zoom speed limit and the flash limiter. Moving a slider plays a small matching effect, and Try it plays a sample. To restyle it, use Copy to Project on the menu: FeelKit's own copy is replaced by each update.	Lower third: Show Feel Comfort Menu	Verify the right-click entry's label on the menu asset
9	5:30-6:05	Project Settings	Show Apply Comfort To Engine Camera Shakes and Apply Comfort To Engine Force Feedback	Comfort also scales the engine's own camera shakes and controller vibration, so effects your game already has follow the same settings. Both options are on by default.	None	
10	6:05-6:35	Debugger beside the game	Tools > FeelKit Debugger ; press 7 in the game; expand Player 0 comfort and Controller vibration	In FeelKit Pro, the Debugger shows each player's comfort settings and what really reaches the controller: the comfort scale and the controller scale.	PRO Label, top right	

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
11	6:35-6:55	Message Log	Tools > FeelKit Comfort Audit; scroll the FeelKit Comfort Audit page	Tools, FeelKit Comfort Audit checks every recipe and the comfort settings for likely problems, such as more than three flashes in a second or saturated red flashes, and lists them in the Message Log. It helps prepare for an accessibility review; it does not certify a game.	PRO Label, top right	
12	6:55-7:05	End card	None	Chapter 8 of the manual lists every comfort setting with its range and default.	End card with the playlist	

8 Listing videos

IN THIS CHAPTER

8.1 T1: FeelKit Pro

8.2 T2: FeelKit Lite

No narration: game sound and short on-screen text carry the message, because Fab visitors may watch without sound. Every frame is FeelKit running in Unreal; nothing is added in the edit except the text and the cuts (Fab requires media to show the real product). The comparison rules in the editing chapter apply to every off / on pair.

8.1 T1: FeelKit Pro

Item	State
Source	The four demo levels in GameFeelDev (Lvl_Combat , Lvl_Platforming) and FeelDemoFP (Lvl_Shooter , Lvl_Horror) on UE 5.6, as the playable demos ship; the recording project for the editor shots
Capture	2560 x 1440 through VSR, exported at 1920 x 1080. Each pair recorded as one take, pressing Tab between the repeats (or with the automated input described in the last chapter)
Export	1920 x 1080, 60 fps, H.264, 16 Mbps: about 150 MB for 75 s, under Fab's 300 MB

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
1	0:00-0:06	Action/RPG, badge FEEL: OFF	Light, light, heavy combo on the dummy	None	Same hit. FeelKit off.	Camera 3.5 m behind, level
2	0:06-0:12	Same place and camera, FEEL: ON	Same combo	None	FeelKit on.	Same length as shot 1 within 5 frames
3	0:12-0:22	Platformer, off then on	Jump, air jump, land on the same platform, twice	None	Jumps and landings	5 s each half
4	0:22-0:32	Shooter, off then on	A 1 s rifle burst at the same target	None	Every shot	5 s each half
5	0:32-0:42	Horror, off then on	Sprint down the same corridor into a scare spot	None	Tension	5 s each half; check the flash count
6	0:42-0:55	Recipe editor, HeavyHit copy	Scrub the playhead; resize the Camera Punch track; Play	None	Build it on a timeline. Preview without pressing Play.	Real editor, slowed pointer
7	0:55-1:02	Blueprint node, then one hit in the game	Hold on the Play Feel node 2 s; cut to the hit	None	Play it with one node.	
8	1:02-1:10	Comfort menu in the Action/RPG level	Move Camera shake to 0%; Try it	None	Players choose how much they feel.	

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
9	1:10-1:15	End card	None	None	FeelKit Pro. Unreal Engine 5.6 to 5.8. Playable demos and the manual are linked below.	Cover style: dark background, green accent

8.2 T2: FeelKit Lite

Item	State
Source	The recording project with the Lite package installed (the Lite package is not built yet); V1 footage where it shows only Lite content
Rule	Nothing on screen may be Pro-only: no Recipe Browser, no waveform on sound tracks, no Debugger, no demo levels that use Feel Triggers or Feel Maps
Export	1920 x 1080, 60 fps, H.264, 16 Mbps (about 80 MB)

Shot	Time	On screen	Actions	Narration	On-screen text	Notes
1	0:00-0:10	Third Person level, off then on	Press 1 with FEEL: OFF, Tab, press 1 with FEEL: ON	None	One key. FeelKit off, then on.	From V1 shot 1 material
2	0:10-0:22	Recipe editor with Lite	Add a Camera Punch track; drag its end; Play	None	A timeline editor with a live preview.	Record with the Lite package
3	0:22-0:30	Blueprint and the game	Play Feel node; hit in the game	None	Play Feel from Blueprint or C++.	
4	0:30-0:37	Comfort menu	Reduced motion; Try it	None	The full comfort layer, free.	
5	0:37-0:40	End card	None	None	FeelKit Lite. Free.	

9 Editing

IN THIS CHAPTER

- 9.1 Project and folders
- 9.2 Timeline structure
- 9.3 Zoom-ins
- 9.4 Callouts and text
- 9.5 Captions
- 9.6 Loudness and mix
- 9.7 Music
- 9.8 Feel off and feel on

9.1 Project and folders

- DaVinci Resolve, one project per video. **Project Settings:** timeline resolution equal to the capture (2560 x 1440 or 1920 x 1080), timeline and playback frame rate 60, color science DaVinci YRGB with Rec.709 Gamma 2.4.
- Folders on D:: [D:/FeelKitVideo/Raw](#) (OBS), [Voice](#), [Edit](#) (Resolve media and cache), [Export](#), [Captions](#), [Thumbnails](#), [Music](#). Nothing under [B:/NewUE5Project](#), which is synced.
- MKV recordings: remux to MP4 in OBS (**File > Remux Recordings**) before importing.

9.2 Timeline structure

Track	Content
V1	Screen clips, cut to the narration
V2	Spotlight and marker overlays
V3	Lower thirds and key captions
V4	Title card and end card

Track	Content
A1	Narration
A2	Unreal sound (game and editor)
A3	Music (listing videos only)

Order of work: place the narration clips on A1 in shot order with 0.3 to 0.5 s between sentences; cut the screen clips on V1 to them; then overlays, captions, the mix and the loudness pass.

- Cut waiting (downloads, compiling, restarts) with a straight cut. Where the screen layout changes across the cut, add a 6-frame cross dissolve. Never speed up interface footage; sped-up pointers look edited.
- Never cut while an effect plays. A result stays on screen at least 1.5 s after it ends.
- Finish each sentence before the key press it describes, so the effect's sound is heard clearly.

9.3 Zoom-ins

The pattern for each zoom: full screen for at least 1 s, start the zoom 1 s before the click, hold while the action happens and 1 s after, return. At most one zoom every 10 s. In Resolve: **Inspector** > **Transform** > **Zoom** and **Position** keyframes with ease in and out.

Subject	Scale (1440p source)	Scale (1080p source)	Zoom in	Zoom out
A small control: check box, pin, dropdown entry, curve key	1.33x	1.25x	12 frames (0.2 s)	12 frames
A panel area: Details category, a Feel Map row, a message	1.25x	1.15x	15 frames (0.25 s)	15 frames
Game view during an effect	Never	Never		

Gameplay is never zoomed or reframed: a zoom changes the apparent size of camera shakes and punches.

9.4 Callouts and text

The same look as the manual's figures, drawn once as transparent PNG overlays at the timeline's resolution (sizes below are for 1920 x 1080; multiply by 1.33 at 1440p).

Element	Specification
Spotlight	Everything outside the subject covered by black 0A0C10 at 40% opacity; the subject inside a rounded rectangle, 3 px outline 2E8B57, corner radius 10 px, 6 px padding. Fades in and out over 8 frames
Numbered marker	Circle 36 px across, fill 2E8B57, 3 px white ring, white number in Segoe UI Bold 22 px

Element	Specification
Lower third (menu paths, node names)	Segoe UI Semibold 30 px, white, on 161B22 at 90% opacity, corner radius 12 px, padding 24 px, 60 px from the bottom left corner; 3 to 5 s on screen, 8-frame fades
Key caption	Same box, key names in Consolas 30 px ("Tab", "Ctrl + D"), bottom center, from the key press for 1.5 s
PRO label	"PRO" in Segoe UI Semibold 20 px, 2E8B57 on EAF5EF with a 2 px 2E8B57 outline, top right, 40 px from the edges, for the whole Pro segment
Title card	Background gradient 0E1116 to 161B22 as on the cover; title Segoe UI Semibold 64 px white; subtitle Segoe UI 32 px 3FB973; a 12 px by 120 px 3FB973 bar above the title
End card	Same background; next video title, the manual link, Discord; 5 s at least

No moving arrows, no sound effects on callouts (FeelKit's sounds must be the only effects the viewer hears), no text over the game view during an effect.

9.5 Captions

- The narration column of each script is the transcript. After the edit, collect the final narration text in order (the developer can extract it from this document).
- YouTube Studio > **Subtitles** > **Add language** (English) > **Add** > **Auto-sync**, paste the text. YouTube sets the timing from the audio. Check it once, correct any caption that starts before its words, then download the file as SRT to <D:/FeelKitVideo/Captions/V1.en.srt>.
- Caption rules: one or two lines, at most 42 characters per line, 1 to 7 s each, UI labels spelled as on screen.
- The listing videos carry their text in the picture and have no captions track.

9.6 Loudness and mix

- **Narration (A1)**: high-pass at 80 Hz; a de-esser only if "s" sounds are sharp; no further compression when the OBS compressor was on.
- **Unreal sound (A2)**: 10 dB below the narration while speech runs; during result shots without speech, as loud as the narration. Every result shot must have its sound.
- **Target**: -14 LUFS integrated over the whole video, true peak at or below -1 dBTP. In Resolve: select all clips > **Normalize Audio Levels**, ITU-R BS.1770-4, Target Loudness -14 LUFS, True Peak -1 dBTP, **Relative**; then check the whole timeline with the Fairlight **Loudness** meter.
- **Why -14**: YouTube lowers louder uploads to about -14 LUFS and does not raise quieter ones, so a quieter video plays quieter than the one before it.
- Listing videos use the same target.

9.7 Music

- **Tutorials:** no music. The narration must stay clear, and FeelKit's own sounds are part of what the viewer judges.
- **Listing videos:** music is optional, mixed so that hits and landings stay at least 12 dB louder than the music.
- **Licenses:** CC0, or a paid license that allows commercial use on YouTube and outside it (the Fab gallery file). No NC (non-commercial) or ND (no derivatives: cutting and looping is a derivative) licenses. No tracks licensed for YouTube only. CC-BY is unsuitable for the Fab file, which has no place for a credit.
- Keep the license text or receipt in <D:/FeelKitVideo/Music> and record the track, source and license in the log. After upload, check YouTube Studio's copyright check before publishing.

9.8 Feel off and feel on

Rules for a fair comparison:

1. Same build, level, graphics settings, camera distance and angle. Only the Feel Switch changes.
2. The same action at the same place. Best: one continuous take in which the action repeats, with Tab pressed between the repeats, so the camera and lighting cannot differ. For identical input timing, use the automated input in the last chapter.
3. The in-game FEEL: ON / FEEL: OFF badge stays visible in every comparison shot. No second label that could disagree with it.
4. No edit effects on either side: no added shake, zoom, flash, speed change, color correction or sound, and the same audio level on both. The off side keeps the game's own sounds.
5. Off first, then on, each 3 to 6 s, equal lengths within 5 frames.
6. Split screen only after the sequential pair: each half is the center 960 px of its frame (cropped, not scaled, so camera motion keeps its size), aligned on the frame where the action connects. The on side then runs longer by the hitstop; do not stretch time to hide it.
7. Both sides at 60 fps with the game's motion blur.

10 Export and publishing

IN THIS CHAPTER

- 10.1 Export settings
- 10.2 YouTube upload
- 10.3 Where the links go

10.1 Export settings

Setting (Resolve Deliver page)	Tutorials (YouTube)	Listing videos (Fab and YouTube)
Format and codec	MP4, H.264, High profile	MP4, H.264, High profile
Resolution	2560 x 1440 when recorded at 1440p or 4K, otherwise 1920 x 1080	1920 x 1080
Frame rate	60	60
Bitrate	Restrict to 24 Mbps at 1080p, 40 Mbps at 1440p (YouTube recommends 12 and 24 Mbps for 60 fps; the rest is headroom for YouTube's re-encode)	16 Mbps
Keyframes	Automatic	Automatic
Audio	AAC, 48 kHz, stereo, 320 kbps or higher	Same
Color tags	Rec.709 Gamma 2.4	Same
File name	FeelKit_V1_install_first_recipe.mp4	FeelKit_Pro_listing.mp4, FeelKit_Lite_listing.mp4

10.2 YouTube upload

- **Channel:** not decided yet (an open item in this document's information gaps: channel, video URLs, who records).
- **Visibility:** Unlisted while checking; Public when the manual that links them is published.
- **Title:** at most 70 characters, for example "FeelKit tutorial 1: Install FeelKit and play your first recipe".
- **Settings:** audience "No, it's not made for kids"; category Science & Technology; video language English; Standard YouTube License; embedding allowed. **Altered or synthetic content:** No, when the voice is the person's own and the picture is real screen capture.
- **Chapters:** the list in each video's setup table goes into the description, first line 0:00, at least three chapters, each at least 10 s, in rising order. Correct the times after the edit.
- **Playlist:** "FeelKit tutorials", V1 to V4 in order. End screen on the end card: the next video and the playlist.
- **Thumbnail:** 1280 x 720, PNG or JPG under 2 MB. A real frame from the video (the timeline or the hit), a dark band over the left third with the video number and a title of three or four words in Segoe UI Semibold 88 px, white, and a 3FB973 accent bar. It must read at 320 x 180; check it at that size.

Description template:

```
Installs FeelKit from Fab, copies a recipe from its library, previews it in the
recipe editor and plays it from a Blueprint in Unreal Engine 5.
```

```
Manual (PDF): <link>
FeelKit Pro on Fab: <link>
FeelKit Lite on Fab: <link>
Support on Discord: https://discord.gg/AtJ6RdwaxA
```

```
0:00 The result
0:27 Installing from Fab
...
```

10.3 Where the links go

- **Manual:** the video table of this document (under "Lists read by the build script", marker `table: videos`). Fill the **Link** column of V1 to V4 with the full address (`https://www.youtube.com/watch?v=<id>`) and set **Minutes** to the final length. `build_manual.py` reads that column: the video boxes then print the link instead of "The video will be added after publishing". Rebuild with `python Tools/Manual/build_manual.py --pdf`.
- **Hosted PDF:** replace the Google Drive file as a new version of the same file (**Manage versions** > **Upload new version** in Drive), which keeps the share link, the one behind `DocsURL` and the listings. Verify the menu names in Drive when doing it.
- **Listings:** the playlist link in both descriptions; the T1 YouTube link in the Pro description (Launch document, section 4.1).

- **Fab gallery:** T1 uploaded to the Pro listing, T2 to the Lite listing: 1920 x 1080, up to 300 MB, MP4, MOV or WEBM (Launch document, section 4.2). Whether the Fab form also takes YouTube links, and where a video sits in the gallery order, is not recorded; check in the publishing form.

11 Quality control and schedule

IN THIS CHAPTER

- 11.1 Checks for every video
- 11.2 Checks per video
- 11.3 Production schedule

11.1 Checks for every video

1. Length within 30 s of the target; chapters valid (0:00 first, at least three, each 10 s or longer).
2. Every spoken label matches the screen.
3. No personal data on screen: e-mail addresses, the Windows user name in paths, other projects in the Recent Projects list, account names in the launcher, notifications.
4. No pointer jumps inside a shot; the click highlight stays on for the whole video.
5. Every result is audible and stays on screen at least 1.5 s.
6. Never more than three flashes in any one second of the video.
7. Loudness -14 LUFS (within 1 LU), true peak at or below -1 dBTP.
8. Captions in sync, American spelling, at most 42 characters per line.
9. Title, description, captions and on-screen text follow the writing rules: no em dashes, no hyphenated words, no exclamation marks.
10. Pro segments carry the PRO label; nothing Pro-only appears unlabeled in a Lite and Pro video.
11. Thumbnail readable at 320 x 180.
12. Watched once full screen at 1440p or 1080p, once on a phone.

11.2 Checks per video

Video	Checks
V1	Version 1.0.0 readable in the Plugins window; banner text readable; the copy saved as HeavyHit in the project; start card readable; O opens and closes the comfort menu
V2	No preview labels readable; Damage slider visible with the PRO label; the validation message readable; the Play in PIE result visible with the mouse free
V3	The Damage 90 and 20 hits in the same camera; the Set Feel Parameter dropdown readable; the Feel Map row readable; both landings in the same camera; the replayed play opens in the recipe editor
V4	The substitute vignette shown in the same camera as the flash; the flash limiter label readable; the comfort menu text readable; the two Pro shots labeled
T1	Every pair follows the comparison rules; the badge visible in every pair; file 1920 x 1080 and under 300 MB
T2	Only Lite content on screen; recorded with the Lite package; file under 300 MB

11.3 Production schedule

Order	Work	When	Hours
1	Install OBS, Resolve and PowerToys; turn on VSR; set up the OBS profile; a 2-minute test recording checked in Resolve (sharpness, both audio tracks, loudness)	First	3
2	Recording project, editor settings, layout, snapshot	First	1.5
3	V1 shots 1, 2 and 4 to 18 with the release package installed: picture 2 h, voice 1 h, edit 4 h	Before submission	7
4	V2: picture 3 h, voice 1.5 h, edit 6 h	Before submission	10.5
5	V3: picture 3 h, voice 1.5 h, edit 6 h	Before submission	10.5
6	V4: picture 2.5 h, voice 1 h, edit 5 h	Before submission	8.5
7	T1: demo captures in the four levels 3 h, edit 5 h	Before submission	8
8	T2	When the Lite package exists	3
9	Captions, thumbnails, unlisted uploads, checks	Before submission	4
10	V1 shot 3 (install from Fab), final V1 edit, publish all, links into the video table, manual rebuilt and replaced on Drive	After the listings are live	3
Total			59

V1 cannot show the Fab install before the listings are live, so V1 goes public shortly after launch; V2 to V4 and the listing videos can be public at launch. Recording in the order above also builds the project state each later video starts from.

12 What can be automated

IN THIS CHAPTER

- 12.1 Smooth frames at a fixed time step
- 12.2 Identical takes for the comparisons
- 12.3 Split between the developer and the person recording

The developer can drive the Unreal editor and the demo levels through automation tests and editor Python. The runners in `Tools/Run` already play every demo level with scripted input (`run_arpg.ps1`, `run_arpg_guard.ps1`, `run_platformer.ps1`, `run_shooter.ps1`, `run_horror.ps1`), switch the feel off for measurements (`run_arpg_camera.ps1 -FeelOff`) and save still pictures (`run_feelswitch_ui.ps1`, the DiagFeel picture diagnostics). None of them records video yet. What follows is what that could and could not add.

12.1 Smooth frames at a fixed time step

- A standalone game started with `-game -benchmark -fps=60 -dumpmovie` renders every frame exactly 1/60 s of game time apart and writes one image per frame to the project's `Saved/Screenshots` folder. It runs slower than real time, so no frame is ever dropped. Verify on 5.6: the image format and folder, and that the demo levels' scripted input works outside the editor.
- FeelKit advances its recipes by the engine's frame time (`FApp::GetDeltaTime` in `FeelSubsystem.cpp`), which the fixed step sets to 1/60 s, so recipes play at their true speed in the frames. Only the cooldown check reads the wall clock, so a recipe with a Cooldown may be allowed or blocked differently during a slow capture.
- No sound.** Frame dumping records no audio, and because the capture runs slower than real time, a separate audio recording (the audio mixer's Start Recording Output and Finish Recording Output nodes write a WAV) would not line up with the frames. Game feel needs its

sound, so silent frames suit only close studies of a hit (slow motion, frame by frame), not the listing videos as a whole.

- **Movie Render Queue** renders Level Sequences, not a played game. Gameplay would first have to be recorded with Take Recorder, and FeelKit's camera modifier, post process and time dilation are applied at runtime by the camera manager and the world, which a recorded sequence does not replay. It is not a route for FeelKit footage.
- Turning an image sequence into a video file needs ffmpeg, which is not installed. Resolve can import numbered image sequences directly, so ffmpeg is needed only if the developer is to encode or check files.

12.2 Identical takes for the comparisons

The most useful automation: a "video" mode for the four playthrough runners that plays the level in real time at 60 fps with a fixed camera, runs the same input twice (Feel Switch off, then on) and shows nothing but the game and the badge. The person records it with OBS, so the sound is captured in real time. Both takes then have the same input to the frame, which makes the off / on pairs in T1 fair by construction. Estimated work: about one to one and a half days for the four levels, plus a check that the input stays in sync with the frame rate on this machine.

12.3 Split between the developer and the person recording

Part	Who	Why	What the developer prepares
Narration in all videos	Person	Their own voice. A synthetic voice reads as generated, and YouTube asks for a disclosure of synthetic content	The narration text, word counts and speaking time per shot (printed by the build script)
Hands-on tutorial actions (V1 to V4)	Person	The viewer copies real pointer movement. A scripted editor changes values without menus opening or the pointer moving, which reads as generated	Project snapshots, the recipe copies, the editor layout and the starting state of each video, by script
Recipe editor loops and scrubbing	Person (OBS)	The editor preview runs in real time; there is no fixed-step capture of editor windows	The recipe and the window arrangement
Demo off / on pairs for T1	Both	Input by script, recording by the person in real time for the sound	The runners' video mode (not built)
Silent slow-motion studies of a hit	Developer	Fixed-step frames are exact	A frame-dump runner (not built, about one day)
Overlays, title and end cards, thumbnails	Developer	Static graphics at a known specification, drawn the same way as the manual's cover and figures	PNG files at the timeline's resolution

Part	Who	Why	What the developer prepares
Caption text, chapters, descriptions	Developer	Text taken from the scripts	Text files for pasting into YouTube Studio
Checks of the exported files	Developer, after ffmpeg is installed	Resolution, frame rate, file size, loudness (ffprobe and the ebur128 filter)	A check script (not built)

In short: of about 33 finished minutes, automation can supply the input for roughly one minute of listing footage, a few seconds of silent slow-motion material, and every static graphic and text. About 90% of the running time is hands-on work with narration, which the person records. Automation saves most of its time in preparation: exact starting states, overlays and text, rather than in footage.